

ARQUITECTURA DEL SOFTWARE

DOCUMENTACION DE ARQUITECTURA - SmartSeller POS v2.0

INFORMACION GENERAL

- Identificacion del Sistema:
- Nombre: SmartSeller POS (Point of Sale)
 - Version: 1.0
 - Arquitectura: Cliente-Servidor (Local)
 - Patron: Model-View-Controller (MVC)
 - Tecnologia: Flutter/Dart + SQLite
 - Fecha: Diciembre 2024

ARQUITECTURA GENERAL DEL SISTEMA

Diagrama de Arquitectura de Alto Nivel:



Principios Arquitectonicos:

1. Separacion de Responsabilidades

- Presentacion: Interfaz de usuario y experiencia
- Logica de Negocio: Reglas de negocio y validaciones
- Acceso a Datos: Persistencia y recuperacion de datos

2. Modularidad

- Modulos Independientes: Cada modulo tiene responsabilidades especificas
- Acoplamiento Bajo: Minima dependencia entre modulos
- Cohesion Alta: Funcionalidades relacionadas agrupadas

3. Escalabilidad

- Arquitectura Extensible: Facil adiccion de nuevas funcionalidades
- Patrones de Diseno: Implementacion de patrones probados
- Optimizacion: Rendimiento optimizado para crecimiento

CAPA DE PRESENTACION (PRESENTATION LAYER)

Tecnologia: Flutter Widgets + Material Design 3

Componentes Principales:

1. Pantallas (Screens)

lib/screens/

- login_screen.dart # Autenticacion de usuarios
- dashboard_screen.dart # Panel principal del sistema
- pos_screen.dart # Punto de venta principal
- products_screen.dart # Gestion de productos
- customers_screen.dart # Gestion de clientes
- users_screen.dart # Gestion de usuarios
- reports_screen.dart # Generacion de reportes
- company_config_screen.dart # Configuracion de empresa
- groups_screen.dart # Gestion de grupos
- debug_screen.dart # Herramientas de depuracion

2. Widgets Personalizados

lib/widgets/

- custom_button.dart # Botones personalizados
- custom_text_field.dart # Campos de texto personalizados
- custom_dialog.dart # Dialogos personalizados
- product_card.dart # Tarjeta de producto
- customer_card.dart # Tarjeta de cliente
- report_chart.dart # Graficos de reportes
- loading_widget.dart # Indicadores de carga

3. Navegacion

- Tecnologia: GetX Navigation
- Patron: Route-based Navigation
- Middleware: Autenticacion automatica
- Rutas: Definidas en main.dart

Patrones de UI Implementados:

1. Material Design 3

- Componentes: Cards, Buttons, TextFields
- Colores: Esquema de colores personalizado
- Tipografia: Jerarquia clara de textos
- Espaciado: Sistema de espaciado consistente

2. Responsive Design

- Breakpoints: Adaptacion a diferentes tamanos
- Layouts: Flexibles y adaptativos
- Componentes: Escalables segun dispositivo

3. Accessibility

- Semantica: Etiquetas semanticas apropiadas
- Navegacion: Soporte para lectores de pantalla
- Contraste: Cumplimiento de estandares WCAG

CAPA DE LOGICA DE NEGOCIO (BUSINESS LOGIC LAYER)

Tecnologia: GetX Controllers + Services

Controladores (Controllers):

1. Controladores de Pantalla

```
// Ejemplo: PosController
class PosController extends GetxController {
  // Estado reactivo
  var cartItems = [].obs;
  var total = 0.0.obs;
  var isLoading = false.obs;

  // Metodos de negocio
  void addProductToCart(Product product, double quantity) {
    // Logica para agregar producto al carrito
  }

  void calculateTotal() {
    // Logica para calcular totales
  }

  void processSale() {
    // Logica para procesar venta
  }
}
```

2. Controladores de Modulos

```
lib/modules/electronic_invoicing/controllers/
■■■■ electronic_invoice_controller.dart # Controlador principal
■■■■ document_generation_controller.dart # Generacion de documentos
■■■■ invoice_status_controller.dart # Estados de factura
■■■■ pending_invoice_queue_controller.dart # Cola de facturas
■■■■ pre_emission_validation_controller.dart # Validaciones previas
```

Servicios (Services):

1. Servicios de Negocio

```
lib/services/
■■■■ auth_service.dart # Autenticacion
■■■■ permissions_service.dart # Gestion de permisos
■■■■ sqlite_database_service.dart # Base de datos
■■■■ print_service.dart # Servicio de impresion
■■■■ company_config_service.dart # Configuracion empresa
■■■■ client_service.dart # Gestion de clientes
```

- reports_service.dart # Generacion reportes
- pdf_reports_service.dart # Reportes PDF
- import_service.dart # Importacion datos
- software_configuration_service.dart # Configuracion sistema

2. Servicios de Facturacion Electronica

lib/modules/electronic_invoicing/services/

- audit_service.dart # Auditoria
- backup_service.dart # Respaldos
- document_generation_service.dart # Generacion documentos
- invoice_status_service.dart # Estados factura
- pending_invoice_queue_service.dart # Cola facturas
- pre_emission_validation_service.dart # Validaciones
- system_configuration_service.dart # Configuracion sistema

Middleware:

1. Middleware de Autenticacion

```
class AuthMiddleware extends GetMiddleware {  
  @override  
  RouteSettings? redirect(String? route) {  
    if (!AuthService.to.isAuthenticated) {  
      return const RouteSettings(name: '/login');  
    }  
    return null;  
  }  
}
```

2. Middleware de Invitado

```
class GuestMiddleware extends GetMiddleware {  
  @override  
  RouteSettings? redirect(String? route) {  
    if (AuthService.to.isAuthenticated) {  
      return const RouteSettings(name: '/dashboard');  
    }  
    return null;  
  }  
}
```

CAPA DE ACCESO A DATOS (DATA ACCESS LAYER)

Tecnologia: SQLite + SQLite3 Flutter Libs

Servicio de Base de Datos:

1. SQLiteDatabaseService

```
class SQLiteDatabaseService {  
  static Database? _database;
```

// Inicializacion

```
  static Future initialize() async {  
    _database = await openDatabase(  
      path,  
      version: 1,  
      onCreate: _createTables,  
      onUpgrade: _upgradeDatabase,
```

```
);  
}
```

// Operaciones CRUD

```
static Future insert(String table, Map data) async {  
  return await _database!.insert(table, data);  
}
```

```
static Future<>> query(  
  String table, {  
    String? where,  
    List? whereArgs,  
  }) async {  
  return await _database!.query(table, where: where, whereArgs: whereArgs);  
}  
}
```

Modelos de Datos:

1. Modelos Principales

lib/models/

- user.dart # Usuario del sistema
- product.dart # Producto/Inventario
- customer.dart # Cliente
- sale.dart # Venta
- sale_item.dart # Item de venta
- company.dart # Datos de empresa
- category.dart # Categoria de producto
- group.dart # Grupo de producto
- permission.dart # Permiso del sistema
- authorization_code.dart # Codigo de autorizacion

2. Modelos de Facturacion Electronica

lib/modules/electronic_invoicing/models/

- electronic_document.dart # Documento electronico
- invoice_status.dart # Estado de factura
- pending_invoice_queue.dart # Cola de facturas
- system_configuration.dart # Configuracion sistema
- audit_log.dart # Log de auditoria

Estructura de Base de Datos:

1. Tablas Principales

-- Usuarios del sistema

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE NOT NULL,  
  password TEXT NOT NULL,  
  full_name TEXT NOT NULL,  
  email TEXT,  
  role TEXT NOT NULL,  
  is_active BOOLEAN DEFAULT 1,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Productos/Inventario
CREATE TABLE products (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
description TEXT,
price REAL NOT NULL,
cost REAL,
stock INTEGER DEFAULT 0,
min_stock INTEGER DEFAULT 0,
type TEXT NOT NULL, -- 'unit' o 'weight'
category_id INTEGER,
group_id INTEGER,
barcode TEXT,
is_active BOOLEAN DEFAULT 1,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (category_id) REFERENCES categories(id),
FOREIGN KEY (group_id) REFERENCES groups(id)
);
```

```
-- Clientes
CREATE TABLE customers (
id INTEGER PRIMARY KEY AUTOINCREMENT,
document_type TEXT NOT NULL,
document_number TEXT UNIQUE NOT NULL,
full_name TEXT NOT NULL,
email TEXT,
phone TEXT,
address TEXT,
city TEXT,
department TEXT,
is_active BOOLEAN DEFAULT 1,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Ventas
CREATE TABLE sales (
id INTEGER PRIMARY KEY AUTOINCREMENT,
customer_id INTEGER,
user_id INTEGER NOT NULL,
total REAL NOT NULL,
subtotal REAL NOT NULL,
tax REAL NOT NULL,
payment_method TEXT NOT NULL,
payment_amount REAL NOT NULL,
change_amount REAL DEFAULT 0,
sale_date DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (customer_id) REFERENCES customers(id),
FOREIGN KEY (user_id) REFERENCES users(id)
);
```

```
-- Items de venta
CREATE TABLE sale_items (
id INTEGER PRIMARY KEY AUTOINCREMENT,
sale_id INTEGER NOT NULL,
product_id INTEGER NOT NULL,
```

```

quantity REAL NOT NULL,
unit_price REAL NOT NULL,
total_price REAL NOT NULL,
FOREIGN KEY (sale_id) REFERENCES sales(id),
FOREIGN KEY (product_id) REFERENCES products(id)
);

```

2. Indices para Optimizacion

```

-- Indices para consultas frecuentes
CREATE INDEX idx_products_name ON products(name);
CREATE INDEX idx_products_barcode ON products(barcode);
CREATE INDEX idx_customers_document ON customers(document_number);
CREATE INDEX idx_sales_date ON sales(sale_date);
CREATE INDEX idx_sales_user ON sales(user_id);
CREATE INDEX idx_sale_items_sale ON sale_items(sale_id);

```

INTEGRACIONES EXTERNAS

1. Servicio de Impresion

Tecnologia: libserialport + Windows Print API

Funcionalidades:

- Impresoras Termicas: Tickets de venta
- Impresoras Laser: Facturas electronicas
- Configuracion Automatica: Deteccion de puertos
- Manejo de Errores: Reconexion automatica

Implementacion:

```

class PrintService {
static final PrintService _instance = PrintService._internal();
factory PrintService() => _instance;
PrintService._internal();

```

```

Future initialize() async {
// Inicializar servicio de impresion
}

```

```

Future printTicket(Sale sale) async {
// Generar y enviar ticket a impresora
}

```

```

Future printInvoice(ElectronicDocument document) async {
// Generar y enviar factura a impresora
}
}

```

2. Integracion con Balanzas

Tecnologia: libserialport + Protocolos estandar

Funcionalidades:

- Comunicacion Serial: RS-232, USB
- Protocolos: Mettler Toledo, Sartorius, Ohaus
- Lectura Automatica: Peso en tiempo real
- Integracion POS: Productos por peso

Implementacion:

```
class ScaleService {  
    static final ScaleService _instance = ScaleService._internal();  
    factory ScaleService() => _instance;  
    ScaleService._internal();  
}
```

```
Future connect(String port) async {  
    // Conectar con balanza  
}
```

```
Future readWeight() async {  
    // Leer peso actual  
}
```

```
Future disconnect() async {  
    // Desconectar balanza  
}  
}
```

3. Servicios DIAN

Funcionalidades:

- Facturacion Electronica: Generacion segun estandares DIAN
- Validacion de Datos: Cumplimiento normativo
- Envio de Documentos: Integracion con servicios DIAN
- Gestion de Estados: Seguimiento de facturas

Implementacion:

```
class DIANService {  
    static final DIANService _instance = DIANService._internal();  
    factory DIANService() => _instance;  
    DIANService._internal();  
}
```

```
Future generateCUFE(ElectronicDocument document) async {  
    // Generar CUFE segun estandares DIAN  
}
```

```
Future sendToDIAN(ElectronicDocument document) async {  
    // Enviar documento a DIAN  
}
```

```
Future checkStatus(String cufe) async {  
    // Verificar estado de factura  
}  
}
```

SISTEMA DE SEGURIDAD

1. Autenticacion

Tecnologia: Hash SHA-256 + Tokens de Sesion

Implementacion:

```
class AuthService extends GetxController {  
    var isAuthenticated = false.obs;  
    var currentUser = Rxn();  
}
```



```

Future login(String username, String password) async {
// Validar credenciales
String hashedPassword = _hashPassword(password);
User? user = await _validateCredentials(username, hashedPassword);

if (user != null) {
  isAuthenticated.value = true;
  currentUser.value = user;
  _generateSessionToken();
  return true;
}
return false;
}

String _hashPassword(String password) {
// Implementar hash SHA-256
}
}

```

2. Autorizacion

Sistema Granular de Permisos:

Roles Predefinidos:

```

enum UserRole {
  admin, // Acceso completo
  seller, // POS y clientes
  maintenance // Solo mantenimiento
}

```

```

class Permission {
  static const String accessPOS = 'access_pos';
  static const String accessProducts = 'access_products';
  static const String accessCustomers = 'access_customers';
  static const String accessReports = 'access_reports';
  static const String accessUsers = 'access_users';
  static const String accessConfig = 'access_config';
  static const String accessElectronicInvoicing = 'access_electronic_invoicing';
}

```

Implementacion:

```

class PermissionsService extends GetxController {
  Map> _rolePermissions = {
    UserRole.admin: [
      Permission.accessPOS,
      Permission.accessProducts,
      Permission.accessCustomers,
      Permission.accessReports,
      Permission.accessUsers,
      Permission.accessConfig,
      Permission.accessElectronicInvoicing,
    ],
    UserRole.seller: [
      Permission.accessPOS,
      Permission.accessCustomers,

```

```

Permission.accessReports,
],
UserRole.maintenance: [
Permission.accessProducts,
Permission.accessConfig,
],
};

```

```

bool hasPermission(String permission) {
User? user = AuthService.to.currentUser.value;
if (user == null) return false;

```

```

List userPermissions = _rolePermissions[user.role] ?? [];
return userPermissions.contains(permission);
}
}

```

3. Auditoria

Registro de Actividades:

```

class AuditService {
static final AuditService _instance = AuditService._internal();
factory AuditService() => _instance;
AuditService._internal();

```

```

Future logActivity(String action, Map details) async {
AuditLog log = AuditLog(
userId: AuthService.to.currentUser.value?.id,
action: action,
details: details,
timestamp: DateTime.now(),
ipAddress: await _getLocalIP(),
);

```

```

await SQLiteDatabaseService.insert('audit_logs', log.toMap());
}
}

```

PATRONES DE DISEÑO IMPLEMENTADOS

1. Singleton Pattern

// Servicios unicos del sistema

```

class AuthService extends GetxController {
static final AuthService _instance = AuthService._internal();
factory AuthService() => _instance;
AuthService._internal();
}

```

```

class SQLiteDatabaseService {
static Database? _database;
static SQLiteDatabaseService? _instance;

```

```

static SQLiteDatabaseService get instance {
_instance ??= SQLiteDatabaseService._internal();
return _instance!;
}

```

```
}
```

2. Repository Pattern

```
abstract class ProductRepository {  
    Future<Product> getAllProducts();  
    Future<Product> getProductById(int id);  
    Future<Product> createProduct(Product product);  
    Future<Product> updateProduct(Product product);  
    Future<Product> deleteProduct(int id);  
}
```

```
class SQLiteProductRepository implements ProductRepository {  
    @override  
    Future<Product> getAllProducts() async {  
        List<Product> maps = await SQLiteDatabaseService.query('products');  
        return maps.map((map) => Product.fromMap(map)).toList();  
    }  
}
```

3. Observer Pattern (GetX)

```
class PosController extends GetxController {  
    // Estado reactivo  
    var cartItems = [].obs;  
    var total = 0.0.obs;
```

```
    // Los widgets se actualizan automaticamente cuando cambian estos valores  
    void addToCart(Product product) {  
        cartItems.add(CartItem(product: product, quantity: 1));  
        calculateTotal();  
    }  
}
```

4. Factory Pattern

```
class ProductFactory {  
    static Product createProduct({  
        required String name,  
        required double price,  
        required ProductType type,  
    }) {  
        switch (type) {  
            case ProductType.unit:  
                return UnitProduct(name: name, price: price);  
            case ProductType.weight:  
                return WeightProduct(name: name, price: price);  
        }  
    }  
}
```

5. Strategy Pattern

```
abstract class PaymentStrategy {  
    Future<void> processPayment(double amount);  
}
```

```
class CashPaymentStrategy implements PaymentStrategy {  
    @override  
    Future<void> processPayment(double amount) async {
```

```
// Logica para pago en efectivo
return true;
}
}
```

```
class CardPaymentStrategy implements PaymentStrategy {
    @override
    Future processPayment(double amount) async {
        // Logica para pago con tarjeta
        return true;
    }
}
```

OPTIMIZACIONES Y RENDIMIENTO

1. Optimizaciones de Base de Datos

Indices Estrategicos:

```
-- Indices para consultas frecuentes
CREATE INDEX idx_products_name ON products(name);
CREATE INDEX idx_products_barcode ON products(barcode);
CREATE INDEX idx_customers_document ON customers(document_number);
CREATE INDEX idx_sales_date ON sales(sale_date);
CREATE INDEX idx_sale_items_sale ON sale_items(sale_id);
```

Consultas Optimizadas:

```
// Consulta optimizada para busqueda de productos
Future> searchProducts(String query) async {
    return await SQLiteDatabaseService.query(
        'products',
        where: 'name LIKE ? AND is_active = 1',
        whereArgs: ['%$query%'],
    );
}
```

2. Cache Inteligente

Cache de Productos:

```
class ProductCache {
    static final Map _cache = {};
    static DateTime? _lastUpdate;

    static Future> getProducts() async {
        if (_cache.isEmpty || _shouldRefresh()) {
            await _loadProducts();
        }
        return _cache.values.toList();
    }

    static bool _shouldRefresh() {
        return _lastUpdate == null ||
            DateTime.now().difference(_lastUpdate!).inMinutes > 5;
    }
}
```

3. Lazy Loading

Carga Diferida de Datos:

```
class ProductsScreen extends StatefulWidget {  
  @override  
  Widget build(BuildContext context) {  
    return FutureBuilder(<br>  
      future: ProductService.getProducts(),  
      builder: (context, snapshot) {  
        if (snapshot.connectionState == ConnectionState.waiting) {  
          return LoadingWidget();  
        }  
        return ProductsList(products: snapshot.data ?? []);  
      },  
    );  
  }  
}
```

FLUJOS DE DATOS

1. Flujo de Autenticacion

Usuario → LoginScreen → AuthController → AuthService → Database
↓
Middleware → DashboardScreen

2. Flujo de Venta

POS → PosController → ProductService → Database
↓
CartController → SaleService → Database
↓
PrintService → Impresora

3. Flujo de Facturacion Electronica

ElectronicInvoiceScreen → ElectronicInvoiceController
↓
DocumentGenerationService
↓
DIANService → Servicios DIAN
↓
Database (Auditoria)

COMPATIBILIDAD Y DEPLOYMENT

1. Plataformas Soportadas

Windows (Principal):

- Version: Windows 10/11 (64-bit)
- Compilacion: flutter build windows
- Distribucion: Ejecutable standalone

Android:

- Version: Android 7.0+ (API 24+)
- Compilacion: flutter build apk
- Distribucion: APK firmado

Web:

- Navegadores: Chrome, Firefox, Safari, Edge
- Compilacion: flutter build web

- Distribucion: Archivos estaticos

2. Requisitos del Sistema

Minimos:

- RAM: 4GB
- Disco: 2GB espacio libre
- Procesador: Intel i3 o equivalente
- Sistema Operativo: Windows 10 (64-bit)

Recomendados:

- RAM: 8GB
- Disco: 5GB espacio libre
- Procesador: Intel i5 o superior
- Sistema Operativo: Windows 11 (64-bit)

3. Dependencias Externas

Base de Datos:

- SQLite: Incluido en el paquete
- Drivers: sqlite3_flutter_libs

Impresion:

- Drivers: Especificos del fabricante
- Puertos: USB, Serial, Red

Balanzas:

- Drivers: Especificos del modelo
- Puertos: Serial, USB

TESTING Y CALIDAD

1. Estrategia de Testing

Testing Unitario:

```
void main() {  
  group('ProductService Tests', () {  
    test('should create product successfully', () async {  
      Product product = Product(  
        name: 'Test Product',  
        price: 100.0,  
        type: ProductType.unit,  
      );  
  
      int id = await ProductService.createProduct(product);  
      expect(id, greaterThan(0));  
    });  
  });  
}
```

Testing de Integracion:

```
void main() {  
  group('POS Integration Tests', () {  
    test('should process complete sale', () async {  
      // Setup  
      Product product = await ProductService.createTestProduct();
```

```
Customer customer = await CustomerService.createTestCustomer();
```

```
// Execute
```

```
Sale sale = await PosService.processSale(  
  products: [product],  
  customer: customer,  
  paymentMethod: PaymentMethod.cash,  
);
```

```
// Verify
```

```
expect(sale.id, greaterThan(0));  
expect(sale.total, greaterThan(0));  
});  
});  
}
```

2. Metricas de Calidad

Cobertura deCodigo:

- Objetivo: > 80%
- Herramienta: flutter test --coverage
- Metricas: Lineas, funciones, clases

Analisis Estatico:

- Herramienta: flutter analyze
- Reglas: flutter_lints
- Metricas: Warnings, errors, hints

ESCALABILIDAD Y MANTENIMIENTO

1. Escalabilidad

Horizontal:

- Modulos Independientes: Facil adicon de funcionalidades
- Servicios Desacoplados: Escalabilidad por componente
- Base de Datos: Optimizada para crecimiento

Vertical:

- Optimizacion de Consultas: Indices estrategicos
- Cache Inteligente: Reduccion de carga de BD
- Lazy Loading: Carga diferida de datos

2. Mantenibilidad

Codigo Limpio:

- Nomenclatura: Nombres descriptivos y consistentes
- Documentacion: Comentarios y documentacion tecnica
- Estructura: Organizacion clara de archivos

Patrones de Diseno:

- MVC: Separacion clara de responsabilidades
- Repository: Abstraccion de acceso a datos
- Service Layer: Logica de negocio centralizada

ROADMAP Y FUTURAS MEJORAS

1. Funcionalidades Planificadas

Corto Plazo (3-6 meses):

- Integracion real con servicios DIAN
- Generacion de PDFs mejorada
- Sincronizacion en la nube
- App movil complementaria

Mediano Plazo (6-12 meses):

- Modulo de contabilidad
- Integracion con bancos
- Sistema de lealtad de clientes
- Analisis predictivo

Largo Plazo (1-2 anos):

- Inteligencia artificial para recomendaciones
- Integracion con marketplaces
- Sistema multi-tienda
- API publica para integraciones

2. Mejoras Tecnicas

Arquitectura:

- Migracion a microservicios
- Implementacion de GraphQL
- Cache distribuido (Redis)
- Base de datos distribuida

Rendimiento:

- Optimizacion de consultas avanzadas
- Implementacion de CDN
- Compresion de datos
- Paralelizacion de procesos

SmartSeller POS v2.0 - Documentacion de Arquitectura
Sistema de Punto de Venta Profesional

Fecha de Documentacion: Diciembre 2024

Version: 1.0

Desarrollador: Oscar Mauricio Gonzalez

Licencia: Propietaria

Arquitectura: MVC + GetX + SQLite